

Assignment 1:

Smart Home Energy Monitor – Daily Billing Snapshot

Scenario

You are developing a simple **energy billing module** for a smart home system. Every evening, the system captures the day's electricity usage and calculates the cost, applies a possible discount if energy-saving mode was active, and prints a summary for the homeowner. Your program must simulate one such daily snapshot.

Your task

Write a C++ program that fulfils all the requirements below. Every requirement is marked with the topic it exercises. Read through the entire list before you start coding.

Program Requirements

- Parts of a C++ Program & `#include` Directive**
 - Include the necessary header(s) to use `cout` and the `string` class.
 - Structure your program correctly with a `main()` function.
- Comments**
 - At the very top of your file, add a **multi-line comment** containing:
 - Your name
 - The current date
 - A brief description: "Smart Home Energy Monitor – Daily Billing Snapshot"
 - Use **single-line comments** throughout your code to explain the purpose of important lines or blocks (e.g., discount calculation, scope demonstration, `sizeof` usage).
- Variable, Literals and Assignment Statement / Identifiers**
 - Declare a `std::string` variable named `homeowner` and **initialize** it with the literal "Alex Johnson".
 - Declare an `int` variable named `dayNumber` and **assign** the literal 15 to it **in the same statement** (initialization).
 - Declare a `float` variable named `energyUsageKWh` and **initialize** it with the literal 23.5.
 - Declare a `double` variable named `costPerKWh` and **initialize** it with 0.14567.
 - Declare a `char` variable named `tariffCategory` and **initialize** it with 'A'.
 - Declare a `bool` variable named `energySavingMode` and **initialize** it with `true`.

4. The `cout` Object

- Print a welcome message:
`===== SMART HOME ENERGY MONITOR =====`
- Then print the homeowner's name on a separate line.

5. Arithmetic Operators

- Calculate the **raw total cost** as:
`rawTotal = energyUsageKWh * costPerKWh`
(Use a `double` variable `rawTotal` to hold the result.)
- If `energySavingMode` is `true`, apply a **10% discount** to the raw total:
`finalTotal = rawTotal * 0.9`
- If `energySavingMode` is `false`, `finalTotal` should be equal to `rawTotal`.
- Use the appropriate arithmetic operators.

6. More about Variable Assignments and Initialization

- After the above calculations, **reassign** the `dayNumber` variable to `16` to simulate the **next day's reading**. Print the new `dayNumber` with an appropriate message later, showing that you can change a variable's value after its initial assignment.

7. The `char` Data Type and the `bool` Data Type

- Display the tariff category (e.g., `Tariff Category: A`).
- Print the status of energy-saving mode. Instead of printing `1` or `0`, print `ACTIVE` if it is `true` and `INACTIVE` if it is `false`.
(You may do this using a simple `if/else` – you do **not** need to include any extra headers.)

8. The C++ `string` Class

- Declare a second `std::string` variable named `message` and assign it the value `"Have a nice day!"`.
- Print this message after the billing summary.

9. Floating-Point Data Types

- Print the energy usage with its original `float` value (e.g., `Energy Used: 23.5 kWh`).
- Print the raw total cost and final total cost with their full precision (the default `cout` printing is sufficient).

10. Determining the Size of a Data Type

- Use the `sizeof` operator to print the sizes (in bytes) of the following data types, **each on a separate line**:
`int`, `float`, `double`, `char`, `bool`, and `std::string`.
- Label each output clearly, e.g., `Size of int: ...`

11. Scope

- In `main()`, declare an `int` variable named `powerLimit` and initialise it to `500`.
- Print `powerLimit` **before** entering the inner block.
- Then open a **new block** using curly braces `{ }`.
- Inside that block:
 - Declare another `int` variable, **also named** `powerLimit`, and initialise it to `300`.
 - Print this inner `powerLimit` (with a message like `Inside block - powerLimit: ...`).
- After the closing brace of the block, print the **outer** `powerLimit` again.
- This will demonstrate how the inner variable shadows the outer one and that the outer variable is unchanged.

12. The `cout` Object (final summary)

- Finally, print a complete summary that includes **all** the collected information in a neat format. The summary **must** display at least:
 - Homeowner name
 - Day number (after reassignment)
 - Energy usage
 - Tariff category
 - Energy saving mode status (ACTIVE/INACTIVE)
 - Final total cost
 - The farewell `message`

Expected Sample Output

Your program's output should look similar to the following (the exact byte sizes depend on your system):

```
===== SMART HOME ENERGY MONITOR =====  
Homeowner: Alex Johnson  
--- Size of data types ---  
Size of int: 4  
Size of float: 4  
Size of double: 8  
Size of char: 1  
Size of bool: 1  
Size of string: 32  
--- Variable Scope Demo ---  
Outer powerLimit (before block): 500  
Inside block - powerLimit: 300  
Outer powerLimit (after block): 500  
--- Billing Snapshot ---  
Day number (after reassignment): 16  
Energy used: 23.5 kWh  
Tariff Category: A  
Energy saving mode: ACTIVE  
Raw total cost: 3.41325  
Final total cost: 3.07192  
Have a nice day!
```

Instructions: What You Must Submit

A single `.cpp` file containing your complete program that fulfils **all** the requirements above. Make sure your code compiles and runs without errors. The program must use **only** the topics listed – no advanced features like loops, arrays, or external libraries beyond `<iostream>` and `<string>`.